



Área Mecánica
Sede Puerto Montt

DOCUMENTACIÓN TÉCNICA

Sistema de Solicitudes de Pañol

Aplicación web progresiva para la gestión de préstamos de herramientas y materiales
Documento de traspaso al equipo de práctica profesional

Elaborado por: Diego Henríquez

Carrera: [Completar]

Período de desarrollo: Junio – Julio 2026 (Práctica Profesional)

Supervisor(a): [Completar]

Dirigido a: Equipo de práctica profesional, Área Mecánica

Repositorio: github.com/areamecanicapuertomontt-web/solicitudpa-ol

Fecha: Puerto Montt, octubre de 2026

ÍNDICE

1. INTRODUCCIÓN	4
2. OBJETIVOS	5
2.1 Objetivo general.....	5
2.2 Objetivos específicos	5
2.3 Objetivo de este documento	5
3. DESCRIPCIÓN GENERAL DEL SISTEMA	6
3.1 Usuarios y roles	6
3.2 Flujo de una solicitud	6
3.3 Módulos y páginas	7
4. ARQUITECTURA Y TECNOLOGÍAS	8
4.1 Vista general.....	8
4.2 Stack tecnológico.....	8
5. ESTRUCTURA DEL PROYECTO	10
6. MODELO DE DATOS.....	11
6.1 Tablas.....	11
6.2 Funciones y triggers.....	11
6.3 Seguridad a nivel de filas (RLS).....	12
7. RUTAS API	13
8. NOTIFICACIONES PUSH Y PWA.....	14
8.1 Funcionamiento	14
8.2 Consideraciones	14
9. INSTALACIÓN Y PUESTA EN MARCHA.....	15
9.1 Requisitos	15
9.2 Paso 1: obtener el código	15
9.3 Paso 2: crear la base de datos (Supabase)	15
9.4 Paso 3: notificaciones push (Edge Function)	16
9.5 Paso 4: correo con Brevo.....	16
9.6 Paso 5: crear los usuarios.....	16
9.7 Paso 6: ejecutar en local.....	17
9.8 Paso 7: publicar en Vercel y activar el keep-alive	17
10. VARIABLES DE ENTORNO	18
11. SEGURIDAD Y REVISIÓN PREVIA AL TRASPASO	19
11.1 Buenas prácticas para el nuevo equipo	19
12. OPERACIÓN Y MANTENIMIENTO	21
12.1 Problemas frecuentes	21

13. LIMITACIONES CONOCIDAS Y RECOMENDACIONES	22
13.1 Limitaciones actuales.....	22
13.2 Mejoras sugeridas para la siguiente etapa	22
14. CONCLUSIONES.....	23
15. REFERENCIAS	24
ANEXO A. LISTA DE VERIFICACIÓN DEL TRASPASO	25
ANEXO B. GLOSARIO.....	26

1. INTRODUCCIÓN

El pañol del Área Mecánica de INACAP Sede Puerto Montt presta herramientas, instrumentos y materiales a los alumnos para sus clases prácticas. Antes de este proyecto, el proceso se hacía con formularios en papel: el alumno llenaba una solicitud, la hacía firmar por su docente y la presentaba en el pañol. Era difícil saber en qué estado estaba cada préstamo, quién tenía qué herramienta y qué faltaba por devolver.

Durante la práctica profesional de junio y julio de 2026 se desarrolló el **Sistema de Solicitudes de Pañol**, una aplicación web progresiva (PWA) que digitaliza ese flujo completo. El alumno solicita desde su celular, el docente aprueba con un clic, el pañol entrega validando un código o QR y registra la devolución, y todos reciben notificaciones en tiempo real.

Este documento entrega la información técnica necesaria para que el nuevo equipo de práctica pueda **instalar, operar, mantener y continuar desarrollando** el sistema. Describe la arquitectura, el modelo de datos, la API, la seguridad y, en especial, los pasos para recrear los servicios externos (base de datos Supabase y correo Brevo), que no se traspasan junto con el código.

IMPORTANTE: servicios que el nuevo equipo debe crear

El repositorio de GitHub contiene **todo el código**, pero **no** incluye las cuentas de los servicios en la nube. El nuevo equipo debe crear las suyas:

- **Base de datos (Supabase):** proyecto nuevo, cargado con `supabase/schema.sql` y `supabase/seed.sql`.
 - **Correo (Brevo):** cuenta nueva, configurada como SMTP de Supabase para enviar los correos de recuperación de contraseña.
 - **Hosting (Vercel), claves VAPID** para notificaciones y la **variable del keep-alive** en GitHub.
- El paso a paso completo está en el capítulo 9, Instalación y puesta en marcha.

2. OBJETIVOS

2.1 Objetivo general

Digitalizar el proceso de solicitud, aprobación, entrega y devolución de herramientas y materiales del pañol del Área Mecánica, mediante una aplicación web accesible desde cualquier dispositivo, que entregue trazabilidad y notificaciones en tiempo real a alumnos, docentes y personal del pañol.

2.2 Objetivos específicos

- Permitir que el alumno cree solicitudes desde su celular, asociadas a su asignatura y docente, y que siga su estado.
- Permitir que el docente apruebe o rechace las solicitudes de sus asignaturas, indicando el motivo del rechazo.
- Validar cada entrega en el pañol con un código de 6 dígitos o un QR, generado al aprobar.
- Registrar devoluciones completas y parciales, y alertar cuando falta material.
- Notificar a cada actor con notificaciones push, sin depender del correo.
- Administrar docentes, alumnos y el plan de mantención de equipos del área.

2.3 Objetivo de este documento

Servir como guía técnica de traspaso, para que un equipo nuevo, sin contacto con el desarrollador original, pueda dejar el sistema funcionando en sus propias cuentas y continuar su desarrollo.

3. DESCRIPCIÓN GENERAL DEL SISTEMA

3.1 Usuarios y roles

El sistema tiene cuatro roles. Cada usuario tiene uno solo, guardado en la tabla `perfiles`.

Tabla 1. Roles del sistema

Rol	Quién es	Qué puede hacer
ALUMNO	Estudiante del área	Crear solicitudes, ver el historial y el estado de las suyas, y mostrar el código o QR de retiro.
DOCENTE	Profesor de una asignatura	Ver las solicitudes asignadas a él y aprobarlas o rechazarlas con un motivo.
PANOL	Encargado del pañol	Entregar materiales validando código o QR, registrar devoluciones, reenviar códigos y administrar alumnos, docentes y equipos.
ADMIN	Jefatura o administrador	Todo lo anterior. Además puede aprobar cualquier solicitud, forzar estados, eliminar registros y cambiar roles.

El "director de carrera" no es un rol aparte: es el usuario cuyo correo se configura en la variable `DIRECTOR_CARRERA_EMAIL`, y recibe una alerta cuando una devolución queda incompleta.

3.2 Flujo de una solicitud

Cada solicitud recorre una serie de estados. Los cambios los gatillan las acciones de cada rol:

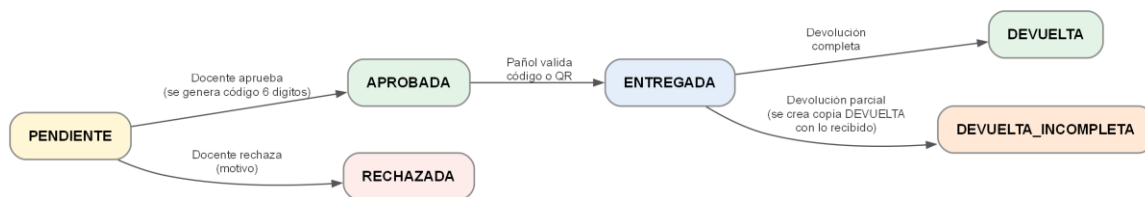


Figura 1. Ciclo de vida de una solicitud (columna `solicitudes.estado`).

- El alumno crea la solicitud** en `/solicitud`. Indica asignatura, sección, jornada, docente y los ítems: cantidad, descripción y si los quiere nuevos, usados o cualquiera. Queda en estado `PENDIENTE` y se notifica al docente.
- El docente decide** desde `/panel`. Si aprueba, el sistema genera un **código de entrega de 6 dígitos**, notifica al alumno y avisa al pañol. Si rechaza, guarda el motivo en `observaciones`.
- El alumno retira** en el pañol mostrando el código o el QR, que ve en "Mis solicitudes". El pañol lo escanea con la cámara o lo digita, y la solicitud pasa a `ENTREGADA`.
- El pañol registra la devolución** con una lista de chequeo por ítem. Si se devuelve todo, la solicitud queda `DEVUELTA`. Si falta algo, la original queda `DEVUELTA_INCOMPLETA` con los ítems pendientes, se crea una copia `DEVUELTA` con lo recibido y se alerta al docente, al alumno y al director de carrera.

3.3 Módulos y páginas

Tabla 2. Páginas de la aplicación

Ruta	Acceso	Descripción
/	Público	Inicio: explica cómo funciona y muestra botones según la sesión.
/login	Público	Inicio de sesión con correo y contraseña. No hay registro abierto: las cuentas las crea el pañol o un administrador.
/olvidar-contrasena /restablecer-contrasena	Público	Recuperación de contraseña por correo (Supabase Auth con SMTP Brevo).
/solicitud	Con sesión	Formulario de solicitud y pestaña "Mis solicitudes" con los códigos QR. Muestra si hay personal de pañol conectado.
/solicitud/[id]/confirmacion	Con sesión	Seguimiento de una solicitud en formato stepper, con el QR al aprobarse.
/aprobar/[token]	DOCENTE, ADMIN	Aprobación directa por enlace con token.
/panel	DOCENTE, PANOL, ADMIN	El docente ve sus pendientes y su historial. El pañol ve todas las solicitudes, filtros, escáner QR, entrega, devolución y reenvío de código.
/admin	PANOL, ADMIN	Dashboard, docentes, alumnos (crear, editar y eliminar cuentas), solicitudes y diagnóstico.
/admin/mantenccion	PANOL, ADMIN	Plan de mantención de equipos (CRUD) y gestión de roles de usuarios.
/equipos	Público	Catálogo de solo lectura de los equipos del plan de mantención.
/qr	Público	QR imprimible que apunta a /solicitud, para pegar en el pañol.

4. ARQUITECTURA Y TECNOLOGÍAS

4.1 Vista general

La aplicación sigue una arquitectura **serverless**. El frontend y las rutas API viven en un mismo proyecto Next.js desplegado en Vercel. La persistencia, la autenticación, el tiempo real y el envío de notificaciones los resuelve Supabase. No hay servidores propios que mantener.

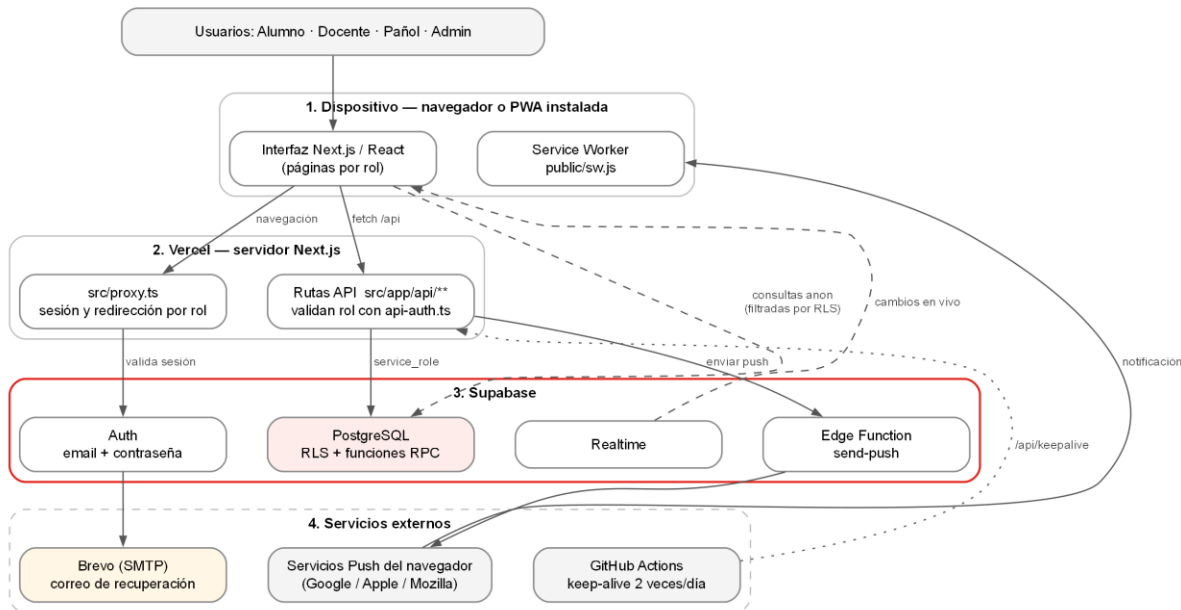


Figura 2. Arquitectura del sistema y comunicación entre componentes.

El navegador consulta directamente la base de datos con la clave pública **anon** para lecturas simples, y las políticas **RLS** (Row Level Security) filtran qué filas ve cada usuario. Las operaciones sensibles (crear solicitudes, aprobar, entregar, devolver y administrar cuentas) pasan por las **rutas API** del servidor. Estas validan la sesión y el rol, y operan con la clave **service_role**.

4.2 Stack tecnológico

Tabla 3. Tecnologías utilizadas

Componente	Tecnología / versión	Uso en el proyecto
Framework	Next.js 16.2 (App Router, Turbopack)	Páginas, rutas API y proxy de sesión.
UI	React 19.2 · TypeScript 5.9	Componentes cliente con tipado estático.
Estilos	Tailwind CSS 4.3 · lucide-react	Diseño responsivo e íconos.
Formularios	react-hook-form 7 · zod 4	Validación del formulario de solicitud.
Base de datos	Supabase · PostgreSQL 17	Tablas, RLS, funciones RPC y triggers.
Autenticación	Supabase Auth (@supabase/ssr 0.10)	Sesión por cookies, login y recuperación.
Tiempo real	Supabase Realtime	Actualiza el panel y la campana de notificaciones.
Notificaciones	Web Push (VAPID) · Edge Function Deno + web-push 3.6	Avisos al celular aunque la app esté cerrada.

Componente	Tecnología / versión	Uso en el proyecto
QR	qrcode · jsQR · BarcodeDetector	Genera el QR del código y lo escanea en el pañol.
Correo	Brevo (SMTP)	Correos de recuperación de contraseña de Supabase Auth.
Hosting	Vercel (+ Analytics y Speed Insights)	Despliegue continuo desde GitHub.
Automatización	GitHub Actions	Keep-alive para que Supabase gratuito no se pause.

Ojo: Next.js 16 no es el Next.js que conocen

Esta versión tiene cambios incompatibles. Por ejemplo, el antiguo `middleware.ts` ahora se llama `proxy.ts`, y `params` y `cookies()` son asíncronos.

Antes de escribir código, revisen la guía incluida en `node_modules/next/dist/docs/`, como indica `AGENTS.md`.

5. ESTRUCTURA DEL PROYECTO

El código está organizado así. El alias `@/` apunta a `src/`.

Tabla 4. Carpetas y archivos principales

Ruta	Contenido
<code>src/proxy.ts</code>	Se ejecuta antes de cada página. Refresca la sesión de Supabase, envía a <code>/login</code> si no hay sesión y redirige según el rol.
<code>src/app/</code>	Páginas del App Router (ver Tabla 2). Cada carpeta es una ruta.
<code>src/app/api/</code>	Rutas API del servidor, una por archivo <code>route.ts</code> (ver capítulo 7).
<code>src/components/</code>	<code>BadgeEstado</code> , <code>SolicitudStepper</code> , <code>NotificationBell</code> , <code>PushHandler</code> , <code>PWAInstallPrompt</code> , <code>UpdatePrompt</code> , <code>HelpButton</code> y <code>Pagination</code> .
<code>src/hooks/useAuthProfile.ts</code>	Hook compartido: muestra el perfil desde el JWT al instante, lo refresca desde perfiles y controla el acceso por rol.
<code>src/lib/supabase-browser.ts</code>	Cliente Supabase único del navegador. <code>supabase-client.ts</code> lo reexporta.
<code>src/lib/supabase-server.ts</code>	<code>createServerClient()</code> , con <code>service_role</code> , y <code>createCookieClient()</code> , con la sesión del usuario.
<code>src/lib/api-auth.ts</code>	<code>requireAuth(request, roles)</code> : valida sesión y rol en las rutas API.
<code>src/lib/push.ts</code> / <code>push-server.ts</code>	Suscripción Web Push en el cliente y envío desde el servidor (llama a la Edge Function).
<code>src/lib/types.ts</code> / <code>utils.ts</code>	Tipos del dominio y utilidades: formato de RUT, fechas y etiquetas de estado.
<code>public/sw.js</code> · <code>manifest.json</code>	Service worker (caché y push) y manifiesto PWA.
<code>supabase/schema.sql</code>	Esquema completo de la base de datos: tablas, funciones, RLS y realtime.
<code>supabase/seed.sql</code>	Datos base: 94 asignaturas y el plan de mantención con 61 equipos.
<code>supabase/functions/send-push/</code>	Edge Function (Deno) que envía las notificaciones Web Push.
<code>supabase/templates/</code>	Plantilla HTML del correo de recuperación de contraseña.
<code>scripts/cargar-usuarios.mjs</code>	Carga masiva de usuarios desde CSV (ejemplo en <code>scripts/usuarios.ejemplo.csv</code>).
<code>.github/workflows/keepalive.yml</code>	Cron que llama a <code>/api/keepalive</code> dos veces al día.
<code>.env.example</code>	Plantilla de variables de entorno (sin valores reales).

6. MODELO DE DATOS

La base de datos es PostgreSQL administrada por Supabase. El script `supabase/schema.sql` la crea completa y `supabase/seed.sql` carga los datos iniciales.

6.1 Tablas

Tabla 5. Tablas de la base de datos

Tabla	Propósito	Columnas clave
perfiles	Perfil y rol de cada usuario de Auth. La crea el trigger <code>handle_new_user</code> .	id (= auth.users.id), rol, nombre, rut, email, jornada (D/V), seccion, carrera, last_seen
docentes	Docentes que aparecen en el formulario de solicitud.	id, nombre, email (único), asignatura, activo
asignaturas	Malla por carrera para el selector del formulario.	codigo, nombre, tipo, horas, carrera (IMI/MI), nivel
solicitudes	Cada préstamo solicitado.	alumno, rut, alumno_email, asignatura, seccion, jornada, carrera, estado, docente_id → docentes, token_aprobacion, codigo_entrega, observaciones
items_solicitud	Ítems de cada solicitud.	solicitud_id → solicitudes, cantidad, descripcion, estado_item (NUEVO/USADO/CUALQUIERA), devuelto
plan_mantencion	Encabezado del plan de mantención de equipos.	titulo, fecha, version, objetivo_general
secciones_mantencion	Secciones del plan (Automotriz, Industrial).	plan_id, numero, nombre, responsable
equipos	Equipos del plan y su programación.	seccion_id, nombre, codigo_inventario, nivel_uso, nivel_costo, frecuencia, mes_programado, estado_programado (P/R/C)
actividades_planificacion registros_mantencion	Actividades del plan e historial de mantenciones. Están preparadas, pero la UI todavía no las usa.	plan_id / equipo_id, tipo, estado, tecnico
push_subscriptions	Suscripciones Web Push de cada dispositivo.	user_id, endpoint (único), p256dh, auth
historial_notificaciones	Historial que muestra la campana de notificaciones.	user_id, titulo, mensaje, url, leído

Relación docente–usuario: un docente existe en dos lugares. Primero, como **usuario de Auth** con rol **DOCENTE**, para iniciar sesión. Segundo, en la tabla **docentes**, para aparecer en el formulario. Ambos se vinculan por el **mismo email**. El script de carga de usuarios crea los dos registros automáticamente si el CSV trae la columna **asignatura**.

6.2 Funciones y triggers

Tabla 6. Funciones de base de datos

Función	Tipo	Descripción
<code>handle_new_user()</code>	Trigger	Al crear un usuario en Auth, inserta su fila en perfiles con los datos de user_metadata.

Función	Tipo	Descripción
update_updated_at()	Trigger	Mantiene updated_at en solicitudes, equipos y push_subscriptions.
rol_actual()	Auxiliar RLS	Devuelve el rol del usuario actual leído desde perfiles. Lo usan todas las políticas.
upsert_push_subscription(endpoint, p256dh, auth)	RPC	Guarda o reasigna la suscripción push del dispositivo actual.
registrar_presencia()	RPC	El panel del pañol la llama cada 30 s para marcar last_seen.
panoleros_activos()	RPC	Lista el personal conectado en los últimos 90 s. Se muestra en el formulario del alumno.
email_registrado(email)	RPC (anon)	Indica si un correo existe, para "Olvidé mi contraseña", sin exponer la tabla.

6.3 Seguridad a nivel de filas (RLS)

Todas las tablas tienen RLS activo. Las reglas principales son:

- **solicitudes**: el personal (ADMIN y PANOL) ve todas. El docente ve las asignadas a él, por id o por email. El alumno ve solo las suyas, por email o RUT. Solo el personal puede modificar y solo ADMIN puede eliminar.
- **items_solicitud**: visibles si la solicitud padre es visible.
- **perfiles**: cada usuario ve el suyo y el personal ve todos. Solo ADMIN puede modificarlos, incluido el rol.
- **docentes, asignaturas y equipos**: lectura pública. Escritura solo para el personal (asignaturas, solo ADMIN).
- **push_subscriptions e historial_notificaciones**: cada usuario accede solo a lo suyo.

El rol se obtiene con `rol_actual()` desde la tabla `perfiles` y **no** desde `user_metadata` del JWT, porque el propio usuario puede editar su `user_metadata` desde el navegador. Si el rol se leyera de ahí, cualquier alumno podría declararse ADMIN.

7. RUTAS API

Las rutas viven en `src/app/api/**/route.ts` y se ejecutan en el servidor de Vercel. Las protegidas usan `requireAuth()` de `src/lib/api-auth.ts`, que acepta la sesión por cookie (fetch del mismo origen) o por el header `Authorization: Bearer <token>`.

Tabla 7. Rutas API disponibles

Método y ruta	Rol requerido	Función
POST /api/solicitudes	Con sesión	Crea la solicitud y sus ítems, y notifica al docente y al alumno.
GET /api/solicitudes	ADMIN, PANOL	Lista todas las solicitudes con docente e ítems.
PATCH /api/solicitudes/[id]/decidir	ADMIN o docente asignado	Aprueba (genera código de 6 dígitos) o rechaza con motivo. Notifica al alumno y al pañol.
POST /api/solicitudes/[id]/reenviar-codigo	ADMIN, PANOL, DOCENTE	Reenvía por push el código de retiro al alumno.
GET /api/panel	ADMIN, PANOL, DOCENTE	Solicitudes para el panel. El docente recibe solo las suyas.
PATCH /api/panel/[id]/entregar	ADMIN, PANOL	Valida el código y marca la solicitud como ENTREGADA.
PATCH /api/panel/[id]/devolver	ADMIN, PANOL	Registra una devolución completa o parcial y envía las alertas.
GET / PATCH /api/notificaciones	Con sesión	Historial de notificaciones y marcar como leídas.
GET /api/docentes	Público	Docentes activos para el formulario.
POST /api/admin/docentes	ADMIN, PANOL	Registra un docente.
POST / PUT / DELETE /api/admin/estudiantes	ADMIN, PANOL	Crea, edita y elimina cuentas de alumnos en Auth. Usa la clave DEFAULT_STUDENT_PASSWORD.
GET /api/keepalive	Público	Consulta mínima para mantener activo Supabase. La llama GitHub Actions.

8. NOTIFICACIONES PUSH Y PWA

8.1 Funcionamiento

1. Al iniciar sesión, `PushHandler` registra el service worker `/sw.js`, pide permiso y suscribe el dispositivo con la clave pública VAPID (`NEXT_PUBLIC_VAPID_PUBLIC_KEY`).
2. La suscripción se guarda con la RPC `upsert_push_subscription`. Un usuario puede tener varios dispositivos.
3. Cuando ocurre un evento (nueva solicitud, aprobación, rechazo o devolución), el servidor llama a `enviarPushNotificacion()`, que guarda el aviso en `historial_notificaciones` e invoca la Edge Function `send-push`.
4. La Edge Function firma el mensaje con la clave privada VAPID y lo envía al servicio push del navegador. Las suscripciones vencidas (respuestas 404 o 410) se eliminan solas.
5. El service worker muestra la notificación y, al tocarla, abre la página correspondiente.

8.2 Consideraciones

- En **iPhone**, las notificaciones solo funcionan si la app está **instalada en la pantalla de inicio** (iOS 16.4 o superior). La app muestra una guía para instalarla.
- Cada vez que se cambie `public/sw.js` hay que subir la versión del caché (`pañol-cache-vXX`) y `CURRENT_VERSION` en `PushHandler.tsx`. Al hacerlo, los usuarios ven el aviso de "nueva versión".
- Las rutas `/api/` y las llamadas a Supabase nunca se guardan en caché.

9. INSTALACIÓN Y PUESTA EN MARCHA

Esta es la parte más importante para el nuevo equipo. El código está en GitHub, pero las cuentas de servicios **deben crearse de nuevo** con correos del área o del equipo. Todas tienen plan gratuito suficiente para el uso del pañol.

Tabla 8. Servicios que se deben crear

Servicio	Para qué	Qué se configura
GitHub	Repositorio y keep-alive	Acceso al repo y la variable <code>KEEPALIVE_URL</code> .
Supabase	Base de datos, Auth, Realtime y Edge Function	Proyecto nuevo, <code>schema.sql</code> y <code>seed.sql</code> , función <code>send-push</code> y SMTP.
Brevo	Envío de correos	Remitente verificado y SMTP key. Se conecta dentro de Supabase.
Vercel	Publicar la aplicación	Importar el repo y cargar las variables de entorno.

9.1 Requisitos

- Node.js 20 o superior y npm.
- Git.
- Editor recomendado: Visual Studio Code.
- Cuentas en GitHub, Supabase, Vercel y Brevo.

9.2 Paso 1: obtener el código

```
git clone https://github.com/areamecanicapuertomontt-web/solicitudpa-ol.git
cd solicitudpa-ol
npm install
cp .env.example .env.local
```

9.3 Paso 2: crear la base de datos (Supabase)

1. Crear un proyecto en **supabase.com**. Región sugerida: São Paulo (**sa-east-1**). Guardar la contraseña de la base de datos.
2. Abrir **SQL Editor** → **New query**, pegar el contenido completo de **supabase/schema.sql** y presionar **Run**. Crea las tablas, funciones, políticas y la configuración de realtime.
3. En otra query, pegar y ejecutar **supabase/seed.sql** (asignaturas y plan de mantención). **Ejecutarlo una sola vez.**
4. En **Project Settings** → **API**, copiar en **.env.local** la **Project URL** (**NEXT_PUBLIC_SUPABASE_URL**), la clave **anon** (**NEXT_PUBLIC_SUPABASE_ANON_KEY**) y la clave **service_role** (**SUPABASE_SERVICE_ROLE_KEY**). Si el panel muestra claves nuevas (**sb_publishable_ / sb_secret_**), usar las de la pestaña **Legacy API keys**, porque la Edge Function valida el JWT de **service_role**.

La clave **service_role** es secreta

Esta clave salta todas las políticas de seguridad. Nunca debe ir en código del navegador, en capturas de pantalla ni en el repositorio. Solo va en **.env.local** y en las variables de Vercel.

9.4 Paso 3: notificaciones push (Edge Function)

```
npx web-push generate-vapid-keys
npx supabase login
npx supabase link --project-ref <ID_DEL_PROYECTO>
npx supabase secrets set VAPID_PUBLIC_KEY=<publica> VAPID_PRIVATE_KEY=<privada>
VAPID_SUBJECT=mailto:<correo-del-area>
npx supabase functions deploy send-push
```

Copiar la clave **pública** VAPID en `NEXT_PUBLIC_VAPID_PUBLIC_KEY` de `.env.local`. La clave **privada** queda solo como secreto de Supabase.

9.5 Paso 4: correo con Brevo

El sistema ya **no** envía correos desde su propio código: las notificaciones se hacen por push. Las primeras versiones usaban la API de Brevo y se reemplazaron en junio de 2026. El único correo que sigue existiendo es el de **recuperación de contraseña**, que envía Supabase Auth. El SMTP que trae Supabase por defecto solo entrega a los miembros del equipo del proyecto y tiene un límite muy bajo, así que **es obligatorio configurar Brevo como SMTP propio**:

1. Crear una cuenta en **brevo.com**, idealmente con el correo del área. El plan gratuito permite 300 correos diarios.
2. En **Senders, Domains & Dedicated IPs** → **Senders**, agregar y **verificar** el correo remitente.
3. En **SMTP & API** → **SMTP**, anotar el **login SMTP** y generar una **SMTP key**.
4. En Supabase, ir a **Authentication** → **Emails** → **SMTP Settings**, activar *Custom SMTP* y completar: host `smtp-relay.brevo.com`, puerto `587`, usuario igual al login SMTP, contraseña igual a la SMTP key, remitente igual al correo verificado y nombre "Pañol Área Mecánica INACAP".
5. En **Authentication** → **Emails** → **Templates** → **Reset Password**, pegar `supabase/templates/restablecer-contrasena.html`.
6. En **Authentication** → **URL Configuration**, definir la **Site URL** (URL de Vercel) y agregar `https://<tu-app>.vercel.app/restablecer-contrasena` en **Redirect URLs** (y `http://localhost:3000/restablecer-contrasena` para desarrollo).
7. Probar desde `/olvidar-contrasena` con un usuario real.

9.6 Paso 5: crear los usuarios

En `.env.local` definir `DEFAULT_STUDENT_PASSWORD`, la contraseña temporal de todas las cuentas nuevas. Luego preparar un CSV con el formato de `scripts/usuarios.ejemplo.csv` y ejecutar:

```
node scripts/cargar-usuarios.mjs mis-usuarios.csv
```

Tabla 9. Columnas del CSV de usuarios

Columna	Obligatoria	Valores
email	Sí	Correo institucional del usuario.
nombre	Sí	Nombre completo.

Columna	Obligatoria	Valores
rol	Sí	ALUMNO, DOCENTE, PANOL o ADMIN.
rut	Alumnos	Formato 12345678-9.
jornada	Alumnos	D (diurna) o V (vespertina).
seccion / carrera	No	Carrera: MI, IMI, FME, FMI, FMC, IMC, etc. Define qué asignaturas ve el alumno.
asignatura	Docentes	Si viene, el docente también se registra en la tabla docentes.

El primer usuario **ADMIN** se crea con este script. Después, el pañol puede crear alumnos desde `/admin`. Cada usuario cambia su contraseña desde "Olvidé mi contraseña".

Datos personales

Los CSV con nombres, RUT y correos reales **nunca** se suben a GitHub: el repositorio es **público**. El `.gitignore` ya excluye `*.sql` y la carpeta `datos-privados/`. Guárdenlos fuera del repo o en esa carpeta.

9.7 Paso 6: ejecutar en local

```
npm run dev      # abre http://localhost:3000
npm run build    # compila como en producción (verificar antes de subir cambios)
```

9.8 Paso 7: publicar en Vercel y activar el keep-alive

1. En **vercel.com**, ir a **Add New** → **Project** e importar el repositorio de GitHub.
2. En **Settings** → **Environment Variables**, cargar las mismas variables de `.env.local` (ver capítulo 10).
3. Desplegar. Cada `git push` a `main` vuelve a publicar automáticamente.
4. Actualizar en Supabase la *Site URL* y las *Redirect URLs* con la URL definitiva.
5. En GitHub, ir a **Settings** → **Secrets and variables** → **Actions** → **Variables** y crear `KEEPALIVE_URL` con el valor `https://<tu-app>.vercel.app/api/keepalive`. Supabase gratuito pausa los proyectos tras 7 días sin actividad, y este cron lo evita.

10. VARIABLES DE ENTORNO

Se definen en `.env.local` para desarrollo y en Vercel para producción. La plantilla está en `.env.example`.

Tabla 10. Variables de entorno

Variable	Dónde	Descripción
NEXT_PUBLIC_SUPABASE_URL	App	URL del proyecto Supabase.
NEXT_PUBLIC_SUPABASE_ANON_KEY	App	Clave pública anon. Sus permisos los limitan las políticas RLS.
SUPABASE_SERVICE_ROLE_KEY	App (servidor)	Clave secreta. Solo se usa en las rutas API y en los scripts.
NEXT_PUBLIC_VAPID_PUBLIC_KEY	App	Clave pública VAPID para suscribirse a push.
DEFAULT_STUDENT_PASSWORD	App (servidor)	Contraseña temporal de las cuentas nuevas.
DIRECTOR_CARRERA_EMAIL	App (servidor)	Correo del perfil que recibe las alertas de devolución incompleta.
VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY / VAPID_SUBJECT	Secretos Supabase	Firma de las notificaciones en la Edge Function send-push.
KEEPALIVE_URL	Variable GitHub Actions	URL de /api/keepalive en producción.

11. SEGURIDAD Y REVISIÓN PREVIA AL TRASPASO

Antes del traspaso se revisó el repositorio completo. Se encontraron y corrigieron los siguientes problemas:

Tabla 11. Hallazgos de la revisión y su corrección

Hallazgo	Riesgo	Corrección aplicada
Las rutas <code>/api/admin/estudiantes</code> y <code>/api/admin/docentes</code> no validaban sesión, y usaban <code>service_role</code> .	Crítico: cualquiera podía crear, modificar o eliminar cuentas.	Ahora exigen rol ADMIN o PANOL con <code>requireAuth()</code> .
GET <code>/api/solicitudes</code> devolvía todas las solicitudes sin sesión.	Alto: exponía nombres, RUT y códigos de entrega.	Restringida a ADMIN y PANOL.
POST <code>/api/solicitudes</code> y PATCH <code>/api/panel/[id]/entregar</code> no validaban sesión.	Medio: permitían solicitudes y entregas anónimas.	Exigen sesión (y rol de pañol en el caso de entregar).
GET <code>/api/panel</code> entregaba todo a cualquier usuario con sesión, incluido un ALUMNO.	Alto	Solo personal. El docente se filtra por id o email.
La contraseña inicial de los alumnos estaba escrita en el código.	Medio	Pasa a la variable <code>DEFAULT_STUDENT_PASSWORD</code> .
Scripts de depuración (<code>scratch/</code>) con correos y contraseñas reales versionados.	Alto: el repo es público.	Se retiraron del repositorio y quedaron ignorados.
Los SQL de la base de datos no estaban en el repo, y faltaban funciones y políticas que solo existían en la BD original.	Bloqueante para recrear la BD.	Esquema consolidado y probado en <code>supabase/schema.sql</code> .
RLS basado en <code>user_metadata.rol</code> , que el usuario puede editar.	Alto: escalada a ADMIN.	Las políticas usan <code>rol_actual()</code> desde perfiles.
<code>last_seen</code> nunca se escribía, así que "pañoleros activos" no funcionaba.	Funcional	RPC <code>registrar_presencia()</code> y <code>panoleros_activos()</code> .
Dependencias y archivos sin uso (Firestore, <code>date-fns</code> , <code>assets</code> de plantilla).	Mantenibilidad	Eliminados.

Historial de Git

Los archivos retirados ya no están en la versión actual, pero **siguen en el historial de commits** del repositorio público.

Como la base de datos se recrea, las contraseñas antiguas quedan sin efecto. Aun así, **no reutilicen** ninguna contraseña del equipo anterior.

Si se quiere borrar el historial, la alternativa es crear un repositorio nuevo a partir de la versión actual.

11.1 Buenas prácticas para el nuevo equipo

- Toda ruta API nueva debe empezar con `requireAuth(request, [roles])`.
- Toda tabla nueva debe tener RLS activo y políticas basadas en `rol_actual()`.

- Nunca subir `.env.local`, CSV de alumnos ni archivos `.sql` con datos personales.
- Revisar los cambios con `npm run build` antes de cada `git push` a `main`, porque Vercel publica automáticamente.

12. OPERACIÓN Y MANTENIMIENTO

Tabla 12. *Tareas periódicas*

Cuándo	Tarea	Cómo
Inicio de cada semestre	Cargar los alumnos nuevos y actualizar docentes y asignaturas.	scripts/cargar-usuarios.mjs, /admin y la tabla asignaturas en Supabase.
Cada cambio en sw.js	Subir la versión del caché.	pañol-cache-vXX en sw.js y CURRENT_VERSION en PushHandler.tsx.
Cada 60 días sin commits	Reactivar el cron de GitHub.	GitHub → Actions → Supabase Keep-Alive → Enable workflow.
Mensual	Revisar los avisos de seguridad de Supabase.	Supabase → Advisors (Security y Performance).
Ante errores	Revisar los registros.	Vercel → Logs; Supabase → Logs y Edge Functions → send-push → Logs.

12.1 Problemas frecuentes

Tabla 13. *Diagnóstico de problemas comunes*

Síntoma	Causa probable	Solución
La app carga pero no muestra datos.	El proyecto Supabase está pausado o las variables están mal.	Restaurar el proyecto en Supabase y revisar las variables en Vercel.
No llegan los correos de recuperación.	Falta el SMTP Brevo, el remitente no está verificado o la Redirect URL es incorrecta.	Revisar el paso 9.5 y Brevo → Transactional → Logs.
No llegan las notificaciones push.	Faltan las claves VAPID, la función no está desplegada o en iPhone la app no está instalada.	Revisar el paso 9.4 y los logs de send-push.
El docente no ve sus solicitudes.	Su email en docentes no coincide con el de su cuenta.	Igualar los correos o cargar al docente con el script.
Los usuarios ven una versión antigua.	Caché del service worker.	Subir la versión del caché (ver la tabla anterior).

13. LIMITACIONES CONOCIDAS Y RECOMENDACIONES

13.1 Limitaciones actuales

- **No hay pruebas automatizadas.** Los flujos se probaron manualmente durante la práctica.
- **Deuda técnica de lint:** `npm run lint` reporta alrededor de 100 errores, en su mayoría tipos `any` y reglas de hooks de React. No impiden compilar, pero conviene corregirlos de a poco.
- El cambio de roles en `/admin/mantenccion` lo permite solo ADMIN. Si un PANOL lo intenta, la base de datos lo bloquea, pero la interfaz todavía no muestra un mensaje claro.
- El proxy (`src/proxy.ts`) usa el rol del JWT solo para redirigir. La autorización real la hacen las rutas API y RLS.
- Las tablas `actividades_planificacion` y `registros_mantenccion` existen, pero no tienen pantallas todavía.
- El QR imprimible de `/qr` se genera con el servicio externo `api.qrserver.com`.

13.2 Mejoras sugeridas para la siguiente etapa

- Agregar pruebas end-to-end (por ejemplo, con Playwright) para el flujo solicitud → aprobación → entrega → devolución.
- Crear un inventario real del pañol (stock por herramienta) y descontarlo al entregar.
- Registrar el historial de mantenciones de equipos desde `/admin/mantenccion`.
- Generar reportes y exportarlos a Excel: préstamos por asignatura, material faltante y alumnos con devoluciones pendientes.
- Migrar los cambios de base de datos a `supabase/migrations/` con la CLI de Supabase, en vez de ejecutar SQL a mano.
- Corregir los errores de lint y activar `npm run lint` como verificación en GitHub Actions.

14. CONCLUSIONES

El Sistema de Solicitudes de Pañol cumple el objetivo de digitalizar el ciclo completo de préstamo de herramientas del Área Mecánica. Hoy entrega trazabilidad por estados, validación de entregas por código o QR, control de devoluciones parciales y notificaciones en tiempo real a cada actor.

La arquitectura serverless (Next.js en Vercel y Supabase) permite operarlo sin servidores propios y dentro de los planes gratuitos. A cambio, sus servicios en la nube están ligados a cuentas personales o del equipo, y por eso deben recrearse en cada traspaso.

Con la revisión previa a este documento, el repositorio quedó listo para entregarse: se cerraron las rutas sin autenticación, se retiraron los datos personales del código, se consolidó el esquema de base de datos en un solo script probado y se documentó paso a paso la puesta en marcha. Si el nuevo equipo sigue el capítulo 9, debería tener el sistema funcionando en sus propias cuentas en pocas horas y poder continuar con las mejoras propuestas.

15. REFERENCIAS

Brevo. (2026). *SMTP relay* [Documentación]. <https://help.brevo.com>

MDN Web Docs. (2026). *Push API*. Mozilla.
https://developer.mozilla.org/es/docs/Web/API/Push_API

Meta Open Source. (2026). *React documentation*. <https://react.dev>

Supabase. (2026). *Supabase documentation*. <https://supabase.com/docs>

Supabase. (2026). *Send emails with custom SMTP*.
<https://supabase.com/docs/guides/auth/auth-smtp>

Supabase. (2026). *Row Level Security*.
<https://supabase.com/docs/guides/database/postgres/row-level-security>

Tailwind Labs. (2026). *Tailwind CSS documentation*. <https://tailwindcss.com/docs>

Vercel. (2026). *Next.js documentation*. <https://nextjs.org/docs>

web-push-libs. (2026). *web-push* [Software]. GitHub. <https://github.com/web-push-libs/web-push>

ANEXO A. LISTA DE VERIFICACIÓN DEL TRASPASO

Marquen cada punto a medida que lo completen:

Tabla 14. *Checklist de puesta en marcha*

✓	Tarea	Referencia
☐	Acceso al repositorio de GitHub	9.2
☐	Proyecto Supabase creado y schema.sql ejecutado	9.3
☐	seed.sql ejecutado (asignaturas y equipos)	9.3
☐	Claves VAPID generadas y función send-push desplegada	9.4
☐	Cuenta Brevo con remitente verificado	9.5
☐	SMTP de Brevo configurado en Supabase y plantilla de correo cargada	9.5
☐	Site URL y Redirect URLs configuradas en Supabase	9.5 / 9.8
☐	Usuario ADMIN, personal de pañol y docentes creados	9.6
☐	Alumnos del semestre cargados	9.6
☐	Proyecto publicado en Vercel con todas las variables	9.8 / 10
☐	Variable KEEPALIVE_URL creada en GitHub	9.8
☐	Prueba completa: solicitud → aprobación → entrega → devolución	3.2
☐	Prueba de "Olvidé mi contraseña" y de notificaciones push en celular	8 / 9.5

ANEXO B. GLOSARIO

Término	Definición
PWA	Progressive Web App: sitio web que se puede instalar en el celular como si fuera una app y que recibe notificaciones.
RLS	Row Level Security: reglas de PostgreSQL que deciden qué filas puede leer o modificar cada usuario.
RPC	Función de la base de datos que se llama desde la app con <code>supabase.rpc()</code> .
<code>service_role</code>	Clave de Supabase con acceso total, que salta las políticas RLS. Es solo para el servidor.
anon key	Clave pública de Supabase, sujeta a las políticas RLS.
VAPID	Par de claves que identifica al servidor que envía notificaciones Web Push.
Edge Function	Función serverless (Deno) que se ejecuta en la infraestructura de Supabase.
SMTP	Protocolo de envío de correo. Brevo actúa como servidor SMTP de Supabase Auth.
Service Worker	Script del navegador que funciona en segundo plano: maneja el caché y las notificaciones push.
Keep-alive	Llamada periódica que evita que Supabase pause el proyecto gratuito por inactividad.